Question Paper - Report

10-Oct-2024 00:43:08
ADITYA PANKAJ MATHPAL .
Logout

Back

Question Paper

MANIPAL ACADEMY OF HIGHER EDUCATION

B.Tech III Semester Sessional Examination September 2024

DATA STRUCTURES [CSE 2122]

Marks: 30 **Duration: 90 mins.**

MCQ

Answer all the questions. Section Duration: 20 mins

1) What will be the output of the following C code?

```
#include< stdio.h>
enum exam
{
a=2,b=3.56
};
enum exam s;
void main()
{
printf("%d%d",a,b);
}
```

(0.5)

Comple-time error 2 3 2 3.56 Run-time error

2) How do you declare an array of pointers to integers?

(0.5)

int* arr[]; arr int*[]; int arr*[]; int[]* arr;

3) What is the postfix expression for the corresponding infix expression?

a+b*c+(d*e)^k

(0.5)

abc+*de*+^k abc*+de*k^+ abc+*de*+k+^ abc*+(de)*+k^

4) A normal queue, if implemented using an array of size MAX_SIZE, gets full when?

(0.5)

Front = (rear + 1)mod MAX_SIZE | Front = rear + 1 | Rear = MAX_SIZE - 1 | Rear = front

5) What does the following function do for a given Linked List with the first node as the head?

```
void fun1(struct node* head)
{
  if(head == NULL)
    return;

  fun1(head->next);
  printf("%d  ", head->data);
}
```

(0.5)

Prints all nodes of linked list | Prints alternate nodes of Linked List | Prints all nodes of linked list in reverse order | Prints alternate nodes in reverse order

6) How many pointers must be updated when inserting a node at the end of a singly linked list?

(0.5)

1 2 3 None of the above

7) Which of the following is true about linked list implementation of queue?

In enqueue operation, if new nodes are inserted at the beginning of linked list, then in dequeue operation, nodes must be removed from end. | In enqueue operation, if new nodes are inserted at the end, then in dequeue operation, nodes must be removed from the beginning. | Both 1 and 2 | None of the above

(0.5)

8) The following steps in linked stack results in which kind of operation?. Here the function getnode() is to obtain a new node for the list. (0.5)

```
newnode = getnode();

newnode->data = item;

newnode->link = list;

list= newnode;
```

push operation at the last | push operation at the front | push operation at the middle | Both 1 and 2

9) Analyze the code snippet given below and identify what it does. Here the function new_Node() is to obtain a new node for the list.

```
Node* head = new_Node();

Node* tail = new_Node();

head->next = tail;

tail->next = head;

Node* current = head;

do { current = current->next;

} while (current != head);
```

(0.5)

It creates a doubly linked list. | It creates a circular linked list and traverses it once. | It creates a linear linked list and traverses it twice. | It creates a circular linked list and traverses it indefinitely.

10) Analyze the following C code snippets and identify the correct one to fully erase the dynamically allocated memory to a polynomial represented using a linked list, where ***poly*** is a pointer to the first node of the list.

(0.5)

```
while (poly != NULL) { struct Node* temp = poly; poly = poly->next; free(temp); }
```

```
free(poly);
```

```
delete(poly)
```

```
while (poly != NULL) { struct Node* temp = poly; poly = poly->next; delete(temp); }
```

DESCRIPTIVE

Answer all the questions.

Answer the following questions.

11) Design the C functions to perform the following operations on a polynomial, where the polynomial is of the form: (4)

$P(x)=a_n.x^n + a_{n-1}.x^{n-1} + ...+ a_1.x + a_0$

1. **Insert** a new term (coefficient and exponent) into the polynomial

2. **Add** two polynomials and return the resulting polynomial.

12) Consider the following structure definition in C.

```
struct Student
{
char name[50];
int rollNumber;
float marks;
};
```

(3)

Write a C function **createAndPrintStudents**that dynamically allocates memory for an array ofStudentstructures, initializes the array with student data, and prints the details of each student in the format given below.

Name: name, Roll Number: rollNumber, Marks: marks

13) Justify the need to convert infix expression to either prefix or postfix? Convert the infix expression A+(B-C*D)/E^F^G to its equivalent postfix by filling the below given table.

(The ^ represents the power operator).

Symbol scanned	Operation performed	Stack content	Postfix array

(3)

14) Write a C program to simulate a Car Parking System (CPS) with a maximum capacity of N parking slots, where no more than N-1 cars can be parked at a time. Each car is identified by its registration number and the owner's Aadhar ID. Vehicles enter the parking lot in order of their arrival and leave in the same order. If the parking lot is full when a new vehicle arrives, the system should notify that the parking lot is full. When all vehicles have exited, the system should indicate that the parking lot is empty. Cars can enter the parking lot if space becomes available due to the departure of other vehicles. (3)

15) Consider the following structure definition. (3)

```
struct list1
```

```
{

int data;

struct list1 *next;

};
```

Write the C functions to implement the following operations on the singly linked list of integers with header node.

Insert: To insert a node into the list in sorted order. The parameters of this function are a pointer to the header node and an integer to be inserted.

RemoveDuplicate: To remove the duplicate element from the list. The parameter of this function is a pointer to the header node.

16) Linked lists find applications in the implementation of queues using dynamic memory allocation. To insert an item into the queue, we allocate memory for the new node, store data in it, and add the node at the rear end of the existing queue. To delete an item from the queue, the node at the front of the queue is removed. Assume the typical function calls for ***enqueue()*** and ***dequeue()*** as ***enqueue(&rear, &front, N);*** and ***dequeue(&front, &rear);,*** where N is the element to be added. The structure definition of the node is as given. (3)

```
// Self- referential structure

typedef struct node *queue; struct node{ int data; Nodeptr next; };
```

Develop the functions to implement linked implementation of ***enqueue()*** and ***dequeue()*** operations.

17) Develop a C function that accepts a pointer to an array and the size of the array, and reverses the elements of the array in place. Use pointer arithmetic and the dereferencing operator to access and modify array elements. (2)

18) Discuss how a stack can help in checking if parentheses in an expression are balanced. Provide an example expression and explain the steps involved in using the stack. (2)

19) Discuss a priority queue using an unsorted array. Explain how you would perform insertion and deletion operations for the same. (2)

LM Exam Suite Paperless Digital Exams © 2024 All rights reserved. epm IP: 45.112.144.168 epCloud 1.5

^ Top